This document provides a comprehensive description of FXRND block, covering both its algorithmic foundations and implementation details, and is structured to ensure a clear understanding of the block's functionality, design methodology, and performance characteristics. The glossary and conventions used throughout this document are presented in Annex A. Section 2 provides a high-level description of the block's behavior/algorithm, offering an intuitive understanding of its operation without delving into implementation-specific aspects. These implementation details are thoroughly addressed in Section 3, which describes the block's interface, including configuration parameters, input/output signals, handshake protocols, and the microarchitecture. To evaluate the block's efficiency and functional correctness, Section 4 presents the synthesis results, including PPA metrics obtained from logic synthesis and power analysis. This section also outlines the validation methodology, detailing the testbench structure used to verify functional compliance and ensure alignment with the design specifications.

1 Overview

The FXRND block is a combinational module that implements quantization and overflow methods according to the IEEE 1666-2023 SystemC standard [1] for fixed-point arithmetic. It features configurable parameters that enable the selection of quantization and overflow modes, offering flexibility to be applied in different arithmetic precision requirements.

2 Behavioral Description

The FXRND block supports multiple rounding and overflow modes, including **truncation**, **rounding**, **saturation**, and its variations like **round to zero** and **round to infinity**. This approach enables flexibility in adapting to various numerical precision requirements. The choice of rounding mode directly impacts the accuracy and statistical properties of the processed data, such as the block's final PPA, which is particularly relevant in applications such as digital signal processing (DSP) and machine learning accelerators.

This section also describes how the block interacts with input data, processes rounding operations, and produces output values while adhering to specific quantization constraints. Key functional aspects such as overflow handling, signed and unsigned arithmetic support, and configurable precision are highlighted to give a clear understanding of its intended behavior.

Diagrams, flowcharts, and example scenarios are presented to illustrate the rounding process in different configurations, ensuring a comprehensive understanding of the block operation before diving into implementation details in subsequent sections.

2.1 Overflow Methods

The block supports multiple overflow handling methods for fixed-point arithmetic, ensuring correct behavior when the result exceeds the representable range. These methods are summarized in Table 1, which provides a comprehensive overview of each overflow scenario. The "Description" column of the table offers a high-level explanation of how the block manages data when overflow occurs, including whether values are saturated, wrapped around, or clipped.

The quantization methods supported for fixed-point precision limitations are listed in table

Overflow Mode	Result
Saturation (SAT)	The result number gets the sign bit of the original number. The remaining bits shall get the inverse value of the sign bit. In unsigned saturation, the remaining bits shall be set to 1 (overflow) or 0 (underflow).
Saturation to Zero (SAT_ZERO)	All bits shall be set to zero if an overflow occours.
Symmetrical Saturation (SAT_SYM)	The result number shall get the sign bit of the original number. The remaining bits shall get the inverse value of the sign bit, except the least significant remaining bit, which shall be set to one. In unsigned saturation, the remaining bits shall be set to 1 (overflow) or 0 (underflow).
Wrap-around (WRAP)	All bits except for the deleted bits shall be copied to the result number.

Table 1: Overflow modes description

2.2 Quantization Methods

Following the same idea of section 2.1, the quantization methods for fixed-point arithmetic are listed in table 2. These methods allows precise control of the rounding during arithmetic operation. The "Description" column provides a high-level explanation of each quantization approach. The block's configurable nature enables the selection of different rounding strategies, such as truncation, rounding to the nearest value, or rounding to infinite. This flexibility allows the FXRND block to meet the accuracy and PPA targets for different applications, ensuring proper handling of fractional values while minimizing quantization errors.

3 Functional Description

This section provides a detailed description of the RTL implementation of the FXRNDblock, including its interface signals, configurable parameters, and underlying its microarchitecture. The FXRNDblock is designed to perform fixed-point quantization and overflow operations with high accuracy and

Quantization Mode	Result
Rounding (RND)	Add the most significant deleted bit to the remaining bits.
Rounding to Zero (RND_ZERO)	If the most significant deleted bit is 1, and either the sign bit or at least one other deleted bit is 1, add 1 to the remaining bits. For unsigned numbers, just copy the remaining bits.
Rounding to Infinity (RND_INF)	If the most significant deleted bit is 1, and either the inverted value of the sign bit or at least one other deleted bit is 1, add 1 to the remaining bits. For unsigned numbers, add the most significant deleted bit to the left bits.
Truncation (TRN)	Copy the remaining bits.

Table 2: Quantization modes description

efficiency. The main aspects of its architecture are outlined, including the internal modules, control logic, and the interaction between input and output signals. Additionally, design considerations impacting power, performance and area will be discussed, offering a thorough insight into how the block is structured to meet the functional requirements defined in the previous section.

The block operation is straightforward, and its architecture is divided into two sequential steps: Overflow and Quantization. Figure 1 below illustrates the bit growth along the combinational path. The red squares represent the integer bits (associated with the overflow handling), while the blue squares represent the fractional bits (related to the quantization method).

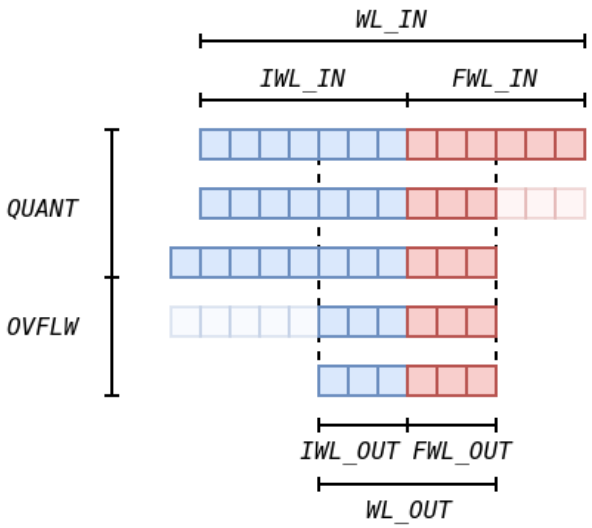


Figure 1: Bit growth through the combinational path.

3.1 Interface Description

The block's interface is presented in Figure 2 while detaild about input and output signals, as well as its configurable parameters set during instanciation are provided in Tables 3 and 4. The behavioral aspects and the block's interaction with other modules in the system will be described later in this section, specifying signal names and direction, as well as the data parallelism.

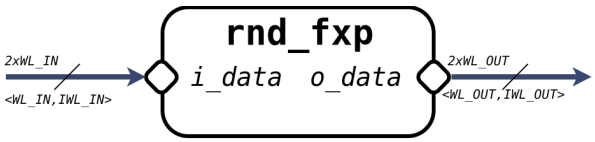


Figure 2: FXRND block interface.

Parameter	Type	Def.	Description
OVFLW	string	'WRAP'	Overflow mode.
QUANT	string	'RND'	Quantization mode.
SIGNED	bit	1	Flag that indicates the number uses two's complement system for signed binary numbers..
WL_IN	uint	8	Input data's wordlength.
IWL_IN	int	2	Input data's integer bits.
WL_OUT	uint	4	Output data's wordlength.
IWL_OUT	int	1	Output data's integer bits.

Table 3: FXRND's interface parameters.

Port	Dir	Par	<WL,IWL*>	Description
i_data	I	1	<WL_IN, IWL_IN>	Input data with initial precision.
o_data	O	1	<WL_OUT, IWL_OUT>	Output quantized/overflowed data.

*Integer wordlength are applicable only for blocks with fixed-point arithmetic.

Table 4: FXRND's interface signals.

3.2 Microarchitecture

N/A.

3.3 Finite State Machines

N/A.

3.4 Timing Diagram

The timing diagram for the FXRND block is relatively simple, as it is a purely combinational block. It illustrates the relationship between input and output signals, demonstrating the immediate propagation of data. The diagram provides some example values using the default operation mode (signed, WRAP + RND), showing the qunatization and overflow modes.

4 Design Metrics and Validation

This section presents the design metrics and validation methodology for the FXRND block. It includes synthesis results, providing detailed information on area utilization, timing performance, and power consumption. Additionally, the verification strategy is described, outlining the test environment, applied stimuli, and coverage metrics used to ensure functional correctness. The goal is to demonstrate that the FXRND block meets the specified design requirements and operates reliably under defined conditions.

4.1 Synthesis Results

TBD

Instance	Leakage	Internal	Switch	Total
TBD	*xx.xxx*	*xx.xxx*	*xx.xxx*	*xx.xxx*

Table 5: FXRND's power results in mW.

Reg2Reg	In2Reg	Reg2Out	In2Out	ClockGate
xx.xxx	*xx.xxx*	*xx.xxx*	*xx.xxx*	*xx.xxx*

Table 6: FXRND's timing(slack) results in ps.

Instance	Count	Cell	Net	Total
TBD	*xxxx*	*xx.xxx*	*xx.xxx*	*xx.xxx*

Table 7: FXRND's Area results in μm^2.

4.2 Validation

The testbench structure is depicted in Figure 3. It consists of three classes (in green) called Driver, Monitor, and Logger. The **Driver** is responsible for generating inputs based on the validation tests, while the **Monitor** captures the DUT's output and compares it with the reference (basically, a reference file). The **Logger** records and prints the test status (pass or fail) for each executed test. The same test task is is present in both Driver and Monitor with different purposes, and the testbench manages their parallel execution.

To run the testbench environment, the user must navigate to the `sandblocks/fxprnd/workspace/` directory and execute the **Makefile** using the command `make run` to perform the RTL simulation. The tests are self-checking and display the status of each stage in the report.

References

[1] "IEEE Standard for Standard SystemC® Language Reference Manual," in IEEE Std 1666-2023 (Revision of IEEE Std 1666-2011) , vol., no., pp.1-618, 8 Sept. 2023, doi: 10.1109/IEEESTD.2023.10246125.

A Glossary and conventions

The table 8 shows the acronyms and abbreviations used in this document.

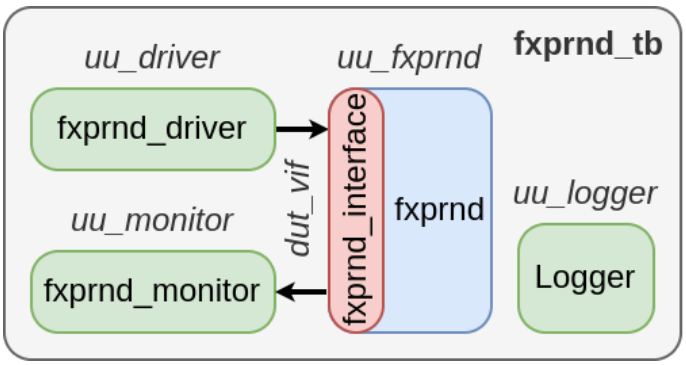


Figure 3: Testbench structure for FXRND.

Term	Meaning
FXRND	Fixed Point Rounder. The block's name.
IWL	Integer Wordlength. Used to specify the number of integer bits of a fixed point data.
LSB	Least Significant Bit.
LSS	Least Significant Sample.
MSB	Most Significant Bit.
MSS	Most Significant Sample.
N/A	Not Applicable. Indicates that the referred info/section is not applicable to the context.
PPA	Power, Performance and Area.
RND	Round. Reffers to fixed-point quantization mode.
RTL	Register Transfer Level.
SAT	Saturation. Reffers to fixed-point overflow mode.
TBD	To Be Done. Used to indicate that the feature/section will be implemented later.
WL	Wordlength. Used to specify the number of bits of a data.

Table 8: Acronyms and abbreviations.

A.1 Fixed-point representation

In some blocks, and behavioral explanations with fixed-point arithmetic there will be used the standard adopted by IEEE 1666-2023 SystemC to represent fixed-point data formats and arithmetic operations. This representation uses the pair $< WL, IWL >$ for representing a sugned/unsigned binary number with WL bits of wordlength and IWL integer bits. This representation gives a range of $\left[0, 2^{WL-IWL} - 1\right] / 2^{WL}$ for unsigned numbers and $\left[-2^{WL-IWL}, 2^{WL-IWL} - 1\right] / 2^{WL}$ for signed numbers.

In some block representations, precision calculations and descriptions involving fixed-point arithmetic, the IEEE 1666-2023 SystemC standard will be used to represent fixed-point data formats and arithmetic operations. This standard defines a number using the pair $< WL, IWL >$, where WL represents the total word length (in bits) and IWL denotes the number of integer bits. The representation range for both signed and unsigned numbers is given by the equations (1) and (2) below:

$$R_{unsigned} = \left[0, 2^{WL-IWL} - 1\right] / 2^{WL} \quad (1)$$

$$R_{signed} = \left[-2^{WL-IWL}, 2^{WL-IWL} - 1\right] / 2^{WL} \quad (2)$$

This notation ensures a consistent and well-defined approach to handling fixed-point arithmetic across different blocks.

A.2 Data representations

Figure 4 shows the color scheme for data representation, bit mapping, and data interconnections used throughout this document.

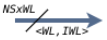
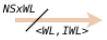
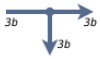
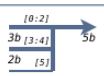
Symbol	Description
	Main datapath data with WL bits and NS parallelism. The corresponding HEX color is #546B8D. If applicable, the fixed-point representation is provided.
	Control signal with NS parallelism. The corresponding HEX color is #00A693. This type of data controls the enable logic of the datapath.
	Static data with WL bits and NS parallelism. The corresponding HEX color is #EDC9AF. If applicable, the fixed-point representation is provided.
	Asynchronous data and signals with NS parallelism. The corresponding HEX color is #893F45.
	**Connection:** The same data is sent to different elements, represented by a connection node at the intersection. In the example, 3-bit data is received and distributed to 3-bit outputs. In this type of data arrangement, the output data remains identical across all connected elements.
	**Derivation:** The received data is divided and sent to different elements. In this representation, a connection node is not used. In the example, 5-bit data is received and distributed to 3-bit, 2-bit, and 1-bit outputs. In this type of data arrangement, the sum of the derived bits is less than or equal to the input data width. The derivated bits can be indicated between brackets [].
	**Concatenation:** The received data is concatenated into a larger word. In this representation, a connection node is not used. In the example, 5-bit data is generated by the concatenation of 3-bit, 2-bit, and 1-bit outputs. In this type of data arrangement, the concatenated wordlength is equal to the sum of input widths. The position of concatenated bits can be indicated between brackets [].

Figure 4: Data representation and connections.

B Revision History

The following table logs the modidications and revisions made in this document, tracking version/revision updates, modification dates, and a brief summary of changes. The Author column lists the contributors responsible for each revision, identified by their initials as follows:

- **TO**: Thiago Morais de Oliveira (`thiagomoraisee@gmail.com`)

Rev	Date	Auth	Description
0.0.1	25/02/20	TO	Initial Draft.
0.1.0	25/03/04	TO	Microarchitecture, behavioral and testbench descriptions.

Table 9: Document revision history.